Apache ZooKeeper™

ECE 454: Distributed Computing

Instructor: Dr. Tahsin Reza
tahsin.reza@uwaterloo.ca

ZooKeeper

Majority of the following slides are based on

the original ZooKeeper paper “Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. 2010. ZooKeeper: wait-free coordination for internet-scale systems. In Proceedings of the 2010 USENIX conference on USENIX annual technical conference (USENIXATC'10). USENIX Association, USA, 11”,

http://www.usenix.org/events/atc10/tech/slides/hunt.pdf,

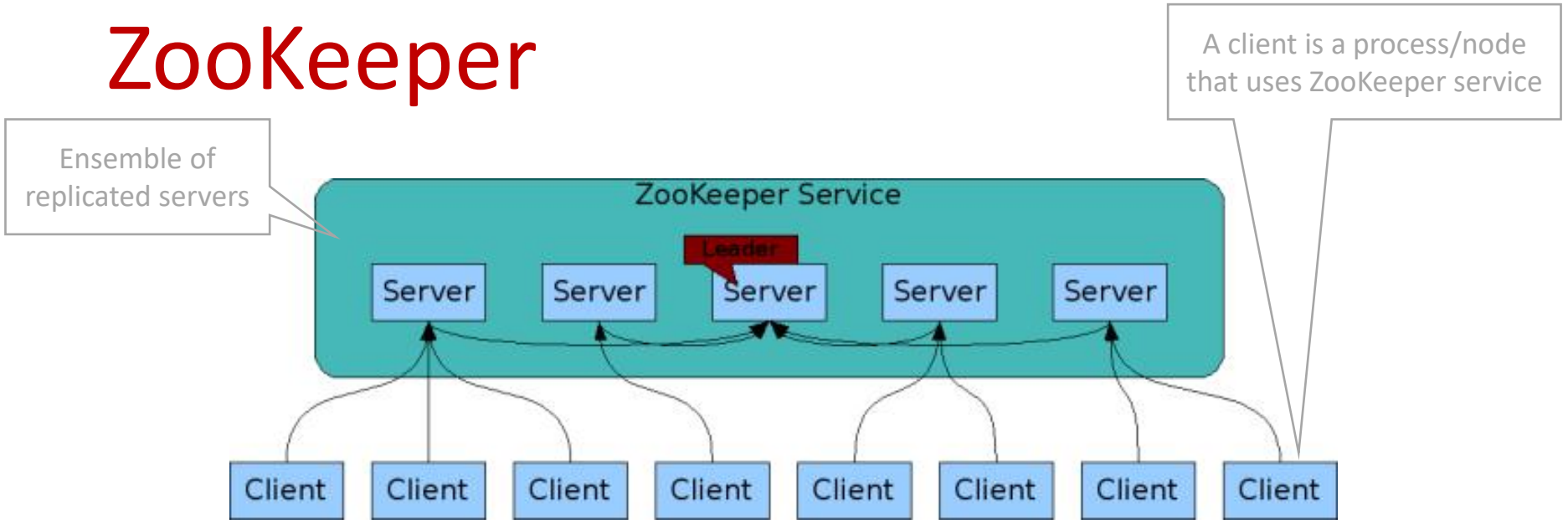
https://zookeeper.apache.org/,

and M. Van Steen and A. S. Tanenbaum, Distributed Systems (textbook).

ZooKeeper

- A distributed coordination service for distributed applications. It exposes a simple set of primitives that applications can build upon to implement higher level services for synchronization, configuration maintenance, and groups and naming.
- Relieves distributed applications the responsibility of implementing coordination services from scratch.
- “Wait-free” - a slow/failed client will not interfere with the request of fast clients.
- Processes coordinate through shared, tree-based hierarchical namespace, akin to that of a filesystem.
- Runs in Java and has bindings for both Java and C (and more languages).
- Offers several guarantees with respect to “availability”. In the distributed computing literature, availability often means that each request eventually receives a correct response.

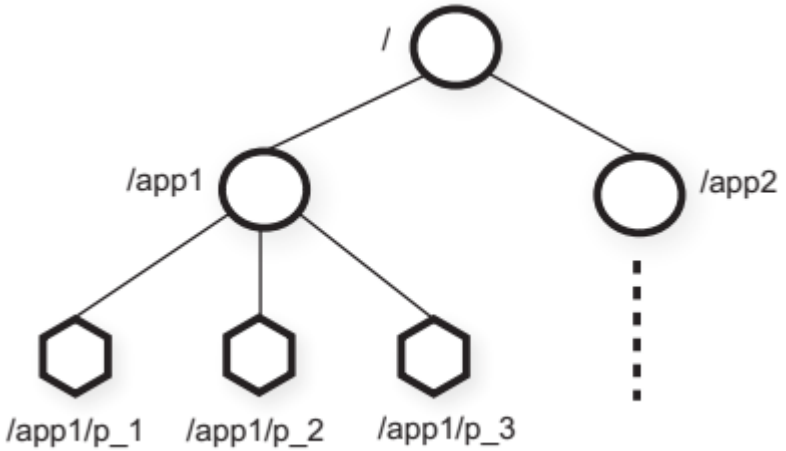
ZooKeeper



- ZooKeeper offers high availability through replicated servers - the server group in a ZooKeeper service is called an **ensemble**.
- A client immediately gets a response from a server.
  - Read requests are handled locally at each server.
  - Writes are handled by the leader (using a voting based approach).
- Section 5.4.3 in the text book describes leader election in a ZooKeeper ensemble which is similar to the bully algorithm.

ZooKeeper: components

- Ensemble
  - Replicated server group
  - Has a leader
- Clients (nodes using ZooKeeper service)
- Sessions
  - A client session persist across servers
- Znode tree (hierarchical namespace)
  - Znode: regular/persistent
  - Znode: ephemeral - can not have children, deleted when the session ends
- Watches (monitoring state change of a znode)
  - Clients receive timely notifications of changes without requiring polling.



Znode tree

ZooKeeper: client API

create(znode_path, data, flags)	creates a node at a location in the data tree
delete(znode_path, version)	deletes a node
exists(znode_path, watch)	verifies if a node exists at a location
getData(znode_path, watch)	reads the data from a node
setData(znode_path, data, version)	writes data to a node
getChildren(znode_path, watch)	retrieves a list of children of a node
sync()	waits for data to be propagated

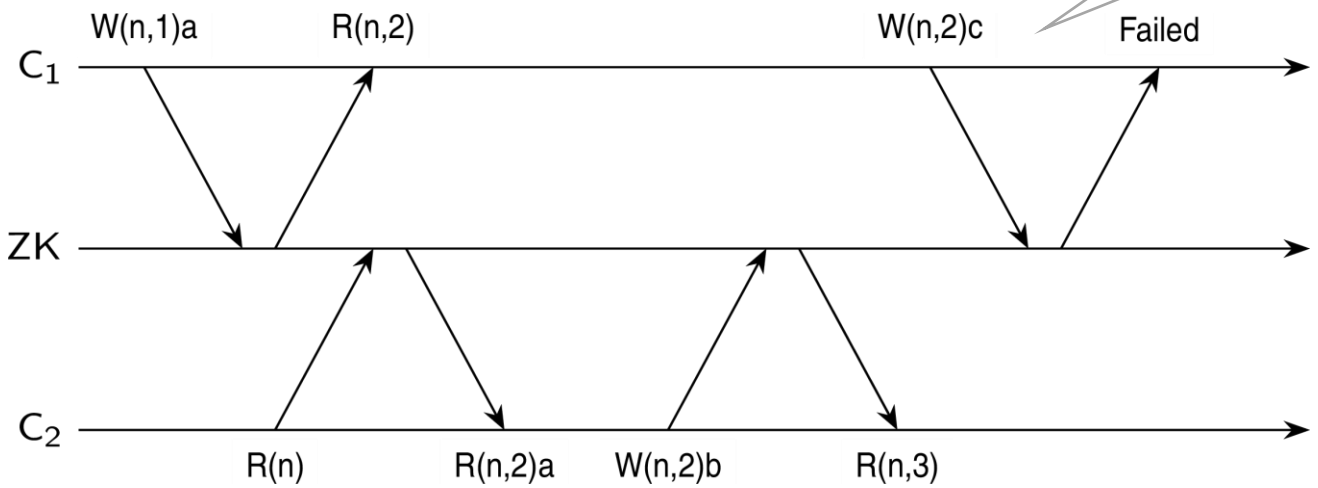
- All methods have both a synchronous and an asynchronous version available through the ZooKeeper API.
- The asynchronous API, however, enables an application to have both multiple outstanding operations and other tasks executed in parallel. The client library guarantees that the corresponding callbacks for each operation are invoked in order.
- Examples: **create**("/znode", data, **SEQUENCE|EPHEMERAL**), **exists**("/znode", **true**).

ZooKeeper: versioning

Not allowed in ZooKeeper

What can C_1 do to perform a successful write?

The goal is to prevent updates based on out-of-date information.



Notation:

- $C_{\#}$ - a client, ZK - ZooKeeper server ensemble
- Write *W*(*n, k*)*a*: request to write *a* to znode *n*, assuming current version is *k*
- Reply *R*(*n, k*): current version of znode *n* is *k*
- Read *R*(*n*): client wants to know the current value of znode *n*
- Reply *R*(*n, k*)*a*: value *a* from znode *n* is returned with its current version *k*

(Section 5.3.6, in the text book)

ZooKeeper: simple lock example

Fully distributed locks that are globally synchronous, meaning at any snapshot in time no two clients think they hold the same lock on a shared resource. Such locking mechanisms can be implemented using ZooKeeper.

1. lock: A client C_1 creates a znode */lock* .
2. lock: A client C_2 wants to acquire the lock but is notified that the associated znode already exists $\Rightarrow$ C_2 subscribes to notification on changes of */lock* (i.e., C_2 watches changes to znode */lock*).
3. unlock: Client C_1 deletes znode */lock* $\Rightarrow$ all subscribers (clients) to changes are notified.

What is the limitation of this design? Can we make it more efficient?

ZooKeeper: leader election example

The goal is to select the client associated with the smallest sequence number as the leader. Let "ELECTION" be a path (znode hierarchical namespace) of choice of the application. To volunteer to be a leader:

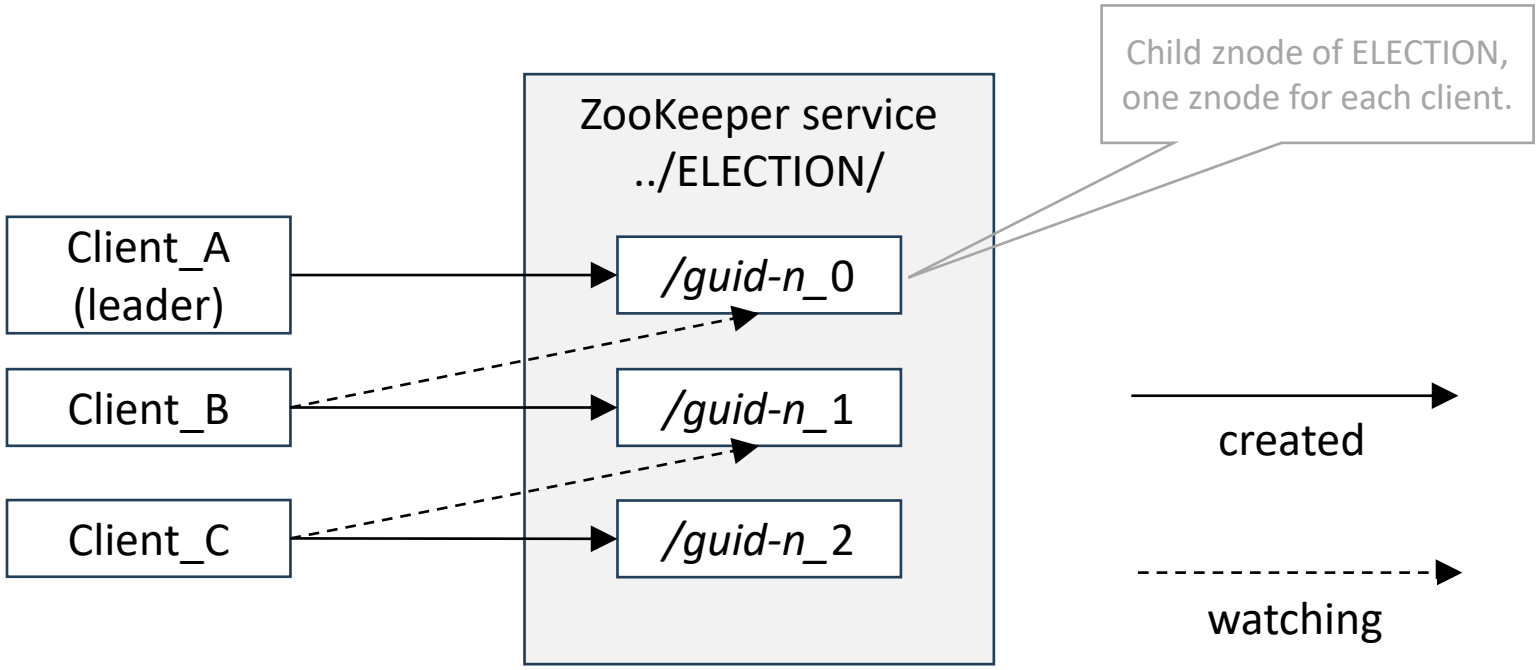
1. Create znode *z* with path "ELECTION/*guid-n_*" with both SEQUENCE and EPHEMERAL flags;
2. Let *C* be the children of "ELECTION", and *i* be the sequence number of *z*;
3. Watch for changes on "ELECTION/*guid-n_j*", where *j* is the largest sequence number such that $j < i$ and *n_j* is a znode in *C.*

Upon receiving a notification of znode deletion:

1. Let *C* be the new set of children of ELECTION;
2. If *z* is the znode with the smallest sequence number in *C*, then execute leader procedure (i.e., the client that created *z* becomes the leader);
3. Otherwise, watch for changes on "ELECTION/*guid-n_j*", where *j* is the largest sequence number such that $j < i$ and *n_j* is a znode in *C.*

ZooKeeper: leader election example

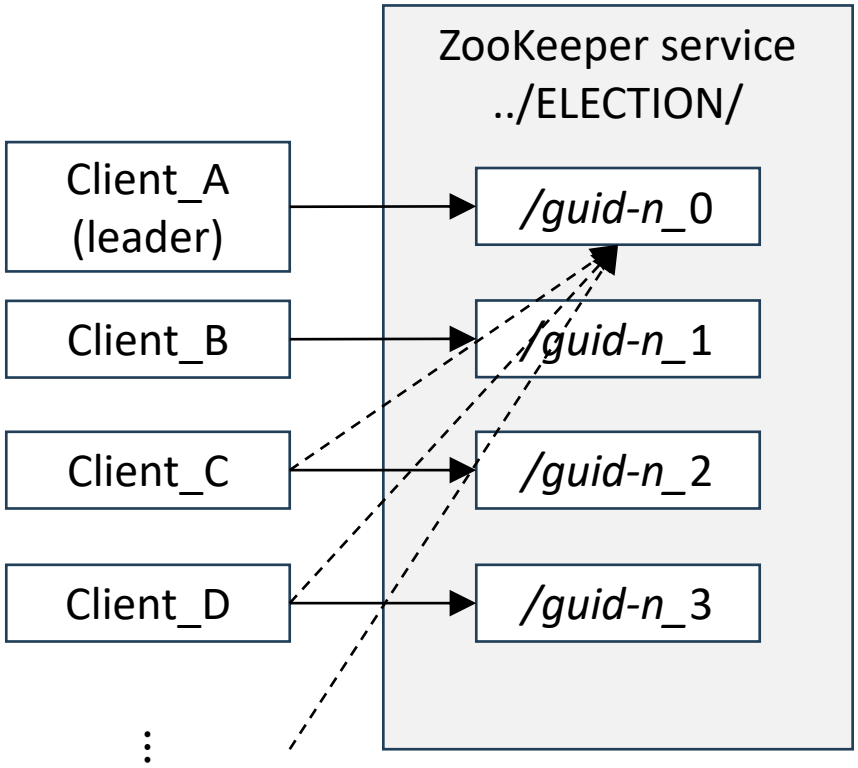
- With the SEQUENCE flag, ZooKeeper automatically appends a sequence number that is greater than anyone previously appended to a child of "ELECTION". The client that created the znode with the smallest appended sequence number is the leader.



ZooKeeper: avoiding herd effect

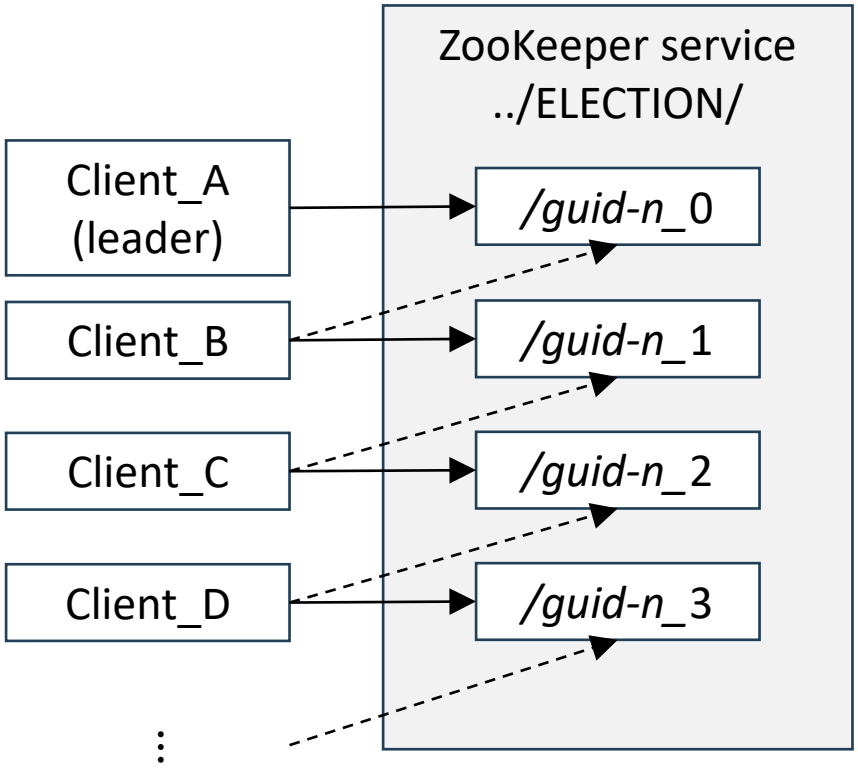
- In leader election, follower clients watch for failures of the leader.
- A trivial solution is to have all application clients watching upon the current znode with the smallest sequence number.
- This causes a **herd effect**: upon a failure of the current leader, all other clients receive a notification, and execute getChildren on "ELECTION" to obtain the current list of children of "ELECTION".
- In the presence of a large number clients, sudden burst of network traffic and increased server load can negatively impact service quality.
- To avoid the herd effect, it is sufficient to watch for the next znode down on the sequence of znodes. If a client receives a notification that the znode it is watching is gone, then it becomes the new leader in the case that there is no smaller znode.

ZooKeeper: avoiding herd effect



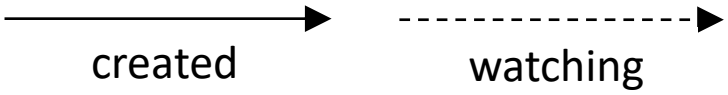
Prone to Herd effect

All watching clients
receive a notification



Improved solution

Only one client
receives a notification



ZooKeeper: guarantees

- Sequential Consistency - Updates from a client will be applied in the order that they were sent (i.e., in FIFO order).
- Linearizable writes - All requests that update the state of ZooKeeper (i.e., data stored on znodes) are serializable and respect precedence.
- Atomicity - Updates either succeed or fail, no partial results. This is enabled by ZooKeeper Atomic Broadcast (ZAB) consensus algorithm.
- Single System Image - A client will see the same view of the service regardless of the server that it connects to.
- Reliability - Once an update has been applied, it will persist from that time forward until a client overwrites the update.
- Timeliness - The clients view of the system is guaranteed to be up-to-date within a certain time bound.

ZooKeeper: guarantees

- From the original ZooKeeper paper appeared in USENIX ATC'10:
  - Write requests from clients are forwarded to the ZooKeeper service leader and must win consensus of majority of the server replicas in the ensemble before response is granted.
  - The server leader executes the request and broadcasts the change to the ZooKeeper state through the ZAB atomic broadcast protocol/consensus algorithm.
  - The server that received the client request, responds to the client when the corresponding state change has been confirmed by the server ensemble.
  - ZAB by default uses simple majority quorums to decide on a proposal, so ZAB and thus ZooKeeper can only work if a majority of servers are correct (i.e., with $2f + 1$ servers ZooKeeper can tolerate f server failures).

Related projects/products

- Google Chubby [1]
- Consul
- etcd
- Redis Lock

[1] Mike Burrows. 2006. The Chubby lock service for loosely-coupled distributed systems. In Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation - Volume 7 (OSDI '06). USENIX Association, USA, 24.